

HiRE: Hindsight Reward Editing for Policy Finetuning

Haoyi Niu^{1*}, Zhengtao Han^{1*}, Yufeng Ji², Zhongyu Li², Koushil Sreenath¹

¹ University of California, Berkeley, ² The Chinese University of Hong Kong

Abstract: Pre-trained robot policies always require finetuning to adapt to specific environments. Reinforcement Learning (RL) offers high performance potential because it improves action optimality rather than simply mimicking data. However, such potential depends heavily on reward quality. Sparse rewards lack process feedback, human-designed rewards are costly and biased, and semantic rewards from foundation representations are often not control-centric. We propose Hindsight Reward Editing (HiRE), a training-free framework to break this reward bottleneck. HiRE bridges the broad knowledge of foundation representation models with physical control awareness, by contrasting successful and failed trajectories in hindsight. It calibrates foundation representation models by identifying “trap states” that are predicted as high-rewarding states yet eventually result in failure, and vice versa. HiRE explicitly penalizes these traps while boosting rewards for critical successful states. This approach can be flexibly compatible with any foundation representations and RL algorithms. Experiments show that HiRE consistently outperforms other reward recipes by delivering dense, control-aware feedback that prevents value function collapse and reward hacking, thereby achieving superior sample efficiency, stable policy updates, and higher performance ceilings, e.g., at least $3\times$ performance of the base policies. Qualitative results at [Project Website](#).

Keywords: Reinforcement Learning, Reward Specification, Policy Finetuning

1 Introduction

Reinforcement Learning (RL) has drawn renewed interest in post-training robot policies [1, 2]. By optimizing for long-term value, RL policies are naturally robust to perturbations and demonstrate strong generalization during deployment. This capability stems from reward signals, which allow RL to ground policies in task objectives rather than merely mimicking expert demonstrations. However, reward specification remains a long-standing bottleneck of robot learning. Sparse rewards fall short on long-horizon tasks because they lack detailed process feedback. To address this, many works leverage human priors to specify dense rewards, which inevitably introduce subjective bias and struggle to generalize across diverse tasks. Unsatisfied with using only task-specific rewards, emergent efforts focus on training expressive reward models that offer fine-grained signals for unseen tasks. Yet, obtaining high-quality reward labels for grounding such large-scale models remains a significant bottleneck. Therefore, current general-purpose reward models still struggle with out-of-distribution (OOD) tasks [3, 4, 5] and often require further calibration to faithfully adapt to new scenarios [6].

These limitations point to two key desiderata for robot reward modeling: generalization across diverse visual scenarios and adaptation to task-specific physical outcomes. For generalization, foundation vision(-language) representations can be directly reused to produce similarity-based rewards in a pretrained latent space [7, 8, 9]. These models are trained on massive amounts of image(-text) data and thus they seldom suffer from OOD issues, while they are not control-aware because they were never trained on robot behavioral data [10, 11]. For adaptation, reward signals should be grounded in physical task outcomes during policy finetuning, using interactive feedback without expensive human labels or additional reward-model training. Thus, a natural question arises: *Can we build a reward scheme that is both universal to generalize and cheap to adapt to new scenarios?* A promising

*Equal contribution, order determined by coin flip and may be listed in either order. Correspondence to: niu@berkeley.edu.

solution would enjoy the broad generalization of foundation vision(-language) representations with minimalist efforts of grounding rewards to task-specific physical outcomes during policy finetuning.

Instantiating such a solution requires addressing two major challenges. Primarily, similarity-based rewards based on foundation representation models often misvalue robot states [12]. For example, it might highly reward a robot for moving toward a placement area even if it has already dropped the object that it should take to that place. This happens because the model prioritizes visually dominant robot motions over small but critical objects, failing to capture the true task objectives. Besides, naïvely adjusting rewards during policy finetuning can destabilize training. A constantly shifting reward landscape often destroys the ability of RL policies to reason about long-term value that helps navigate to final task completion [13]. To address these, we instantiate **Hindsight Reward Editing (HiRE)**, a training-free reward recipe that adjusts rewards produced by foundation vision(-language) representations by contrasting successful and failed trajectories in hindsight of interactions during policy finetuning. HiRE derives a closed-form formulation that directly calibrates reward landscape: it penalizes states that are too close to failure traps and boosts states that lead to success. This approach ensures the rewards are grounded in actual physical outcomes without requiring any extra gradient-based optimization. Furthermore, we incorporate these adjusted rewards through potential-based reward shaping [14]. By combining sparse rewards that indicate success with our refined dense rewards, we provide control-aware feedback while mathematically guaranteeing that optimal policy for task success remains unchanged. These designs allow HiRE to break the control-unawareness bottleneck of foundation representation models with minimal computation and high efficiency.

In summary, our contributions are three-fold: (1) **A practical and scalable reward recipe for real-world robot policies.** We bridge the gap between the strong generalization of foundation representations and the specific task requirements of physical control. By using interaction data from specific tasks, HiRE provides the “missing piece” of control awareness through a training-free reward editing paradigm that empowers sample-efficient RL. It can be highly compatible with any foundation representation models like DINOv2 [10] and SigLIP [11], as well as any RL finetuning algorithmic designs summarized in DICE-RL [15]. (2) **Strong empirical results on simulation and real-world tasks.** HiRE achieves strong performance and high sample efficiency on prehensile, non-prehensile, and long-horizon manipulation tasks on MimicGen [16] and RoboMimic [17] where it can improve low base success rates ($< 10\%$) to near 100%, as well as on a single YAM robot arm where it achieves at least $3\times$ improvement sometimes with only 30 online rollouts. (3) **In-depth analyses on effects of HiRE.** We provide systematic studies to understand why HiRE works, including visualizing the evolving reward landscapes during policy finetuning, and ablation studies on the key components.

2 Related Work

2.1 Reward Learning for Robotics

Learning effective reward models remains a long-standing bottleneck in RL for robotics. One line of work learns task-specific reward models from explicit supervision. Methods such as DrS [18] and SARM [19] require intensive human annotations of stages or task progress, while Rank2Reward [20] leverages rankings from passive videos to learn shaped reward functions. Recent methods further improve trajectory-level reward evaluation: RoboReward [3] and RoboMeter [4] introduce dedicated metrics for trajectory comparison and progress tracking, while Robo-Dopamine [6] learns a reward model from large-scale multi-view manipulation data. Despite their effectiveness, these methods often require task-specific supervision, reward-model training, or large-scale curated robotic data, limiting their generalization to novel visual scenes and task semantics. A complementary line of work derives zero-shot rewards from embedding-space similarity between current and goal observations. Representations such as VIP [8], LIV [9], R3M [21], and DecisionNCE [22] are pretrained on passive behavior datasets so that representation distance reflects task progress. However, since these models can only learn from passive expert data, they often suffer from OOD generalization issues in unseen scenarios. To address this, GCR [12] extends VIP with a goal-contrastive objective trained on task-specific successful and failed trajectories to finetune embeddings during deployment.

To fundamentally bypass such OOD vulnerabilities, emergent efforts repurpose large-scale foundation vision(-language) representations trained on internet-scale, non-robotic data to achieve robust open-world generalization. For instance, CLIPReward [7] extracts language-conditioned rewards from CLIP [23] by evaluating the cross-modal semantic alignment between text prompts and robot visual observations. RewardingDINO [24] injects task-specific semantics by training an explicit preference-based reward head on top of frozen DINOv2 embeddings [10]. TOPReward [5] instead extracts training-free rewards from pretrained vision-language models (VLMs) by scoring task-completion token probabilities given trajectory prefixes and instructions. Although these approaches enjoy exceptional generalization ability of foundation models, they are inherently *control-unaware* due to the lack of physical interaction data in their pretraining phase. They evaluate task progress strictly through passive visual matching, making them highly prone to overestimating deceptive states where visually similar scenes correspond to opposite physical outcomes. HiRE achieves training-free reward editing while remaining control-aware. It calibrates foundation representation models in hindsight by contrasting successful and failed rollouts, which introduces no auxiliary network and no manual dense labels, and is agnostic to choices of representation models and RL algorithms.

2.2 RL Policy Finetuning for Robotics

Recent work has increasingly used RL in post-training stage for pretrained robot policies [25]. One line tailors specialized RL recipes to the structure of generative policies: π_{RL} [26] makes flow policies amenable for direct finetuning by PPO [27] via constructing tractable likelihoods. Direct finetuning is often expensive for fresh samples, so DSRL [28] instead keeps the base policy frozen and only steers in its latent-noise space. Moreover, ResFiT [29] and Policy Decorator [30] attach residual modules that learn additive offsets to the base policy output, whereas RL-Token [2] distinguishes itself from traditional residual methods by training a lightweight policy head for final actions upon a compact token readout of a Vision-Language-Action (VLA) model. DICE-RL [15] instead freezes the base policy and treats it as a stochastic action proposal, contracting its distribution toward high-value modes through a residual actor [29, 30] and value-guided action selection [31]. While all these methods focus on policy improvement, HiRE is entirely orthogonal as it addresses the reward improvement. Consequently, HiRE can be seamlessly integrated with any of these policy finetuning methods, lifting their performance ceilings by providing dynamic, training-free reward shaping that guides policy optimization toward physically correct trajectories.

3 Preliminaries

We formulate the RL problem as a finite-horizon Markov Decision Process (MDP) [32], defined by a tuple $\mathcal{M} := (\mathcal{S}, \mathcal{A}, R, \mathcal{P}, \gamma, H)$. $\mathcal{S}$ and $\mathcal{A}$ denote the state and action space, $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ represents the reward function, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ stands for the transition dynamics, $\gamma \in [0, 1]$ is the discount factor and H is the task horizon. The goal of RL is to find the optimal policy π^* that maximizes cumulative discounted reward starting from an initial state distribution ρ , *i.e.*, $\pi^* = \arg \max_{\pi} \mathbb{E}_{s_0 \in \rho, a \sim \pi(\cdot|s), s' \sim \mathcal{P}(\cdot|s, a)} \left[\sum_{t=0}^H \gamma^t R(s, a) \right]$. RL methods based on approximated dynamic programming typically learn an action-value function $Q(s, a)$, and optionally, a state value function $V(s)$ to practically estimate the cumulative discounted reward for policy optimization.

4 HiRE: A Training-free Reward Recipe for Policy Finetuning

Our objective is to build a control-aware reward function from semantic foundation representations without any additional training. To achieve this, we leverage success and failure signals in past interactions to dynamically reshape the reward landscape in hindsight.

Ideally, an optimal control-aware reward should guide the robot away from failure traps and toward successful paths. In adversarial imitation learning [33, 34], a standard binary cross-entropy objective is widely used to distinguish expert demonstrations from policy rollouts. Drawing inspiration from this, we can analogously formulate a discriminator $D(s)$ to distinguish the success manifold from the failure one. This discriminator is trained to predict the probability that a given state s belongs to a successful trajectory: $\max_D \mathbb{E}_{s \sim p_+} [\log D(s)] + \mathbb{E}_{s \sim p_-} [\log(1 - D(s))]$ where $p_+(s)$ and $p_-(s)$

denote the state distributions of successful and failed states, respectively. According to Maximum Entropy Inverse RL [35], the optimal reward function $R^*(s)$ is mathematically tied to the log-odds of this perfect discriminator $D^*(s)$:

$$R^*(s) = \log \left(\frac{D^*(s)}{1 - D^*(s)} \right) = \log p_+(s) - \log p_-(s) \quad (1)$$

This formulation delivers a key insight: **the ideal reward is simply the log-density ratio between success and failure**. Thus, rather than training a parameterized discriminator through costly optimization, we can analytically determine the optimal reward if these densities can be evaluated directly. This shifts the core challenge from neural network training to density estimation, opening the door for a training-free, non-parametric solution within the representation space.

4.1 Non-parametric Reward Estimation in Foundation Representation Space

To completely circumvent expensive model training, we estimate the densities $p_+(s)$ and $p_-(s)$ using non-parametric Kernel Density Estimation (KDE). Under the general KDE framework, the probability density at any state s is evaluated by averaging the kernel similarities to all historical samples within a support set: $p(s) \approx \frac{1}{N} \sum_{i=1}^N K(s, s_i)$. A major challenge is that frozen vision foundation models produce representations that reside on a high-dimensional unit hypersphere due to L_2 normalization. Traditional Euclidean kernels perform poorly in such spaces because they ignore the curved geometry of the manifold. To respect this geometric property, we employ the Von Mises-Fisher (vMF) kernel [36], which is specifically designed for directional data on hyperspheres:

$$K(s, s_i) = C(\kappa) \exp(\kappa \cdot \text{sim}_\phi(s, s_i)) \quad (2)$$

where $\text{sim}_\phi(s, s_i)$ represents the cosine similarity function in the foundation representation space ϕ ; κ is the concentration parameter, and $C(\kappa)$ is a normalization constant. By applying this kernel to the success buffer $\mathcal{B}^+$ and the failure buffer $\mathcal{B}^-$, we can directly formulate the log-densities. By embedding the non-parametric expectation within the logarithm and separating the multiplicative constants into additive ones, we obtain:

$$\log p_+(s) = \log [C(\kappa^+) \mathbb{E}_{\mathcal{B}^+} \exp(\kappa^+ \text{sim}_\phi(s, g^+))] = \log \mathbb{E}_{\mathcal{B}^+} \exp(\kappa^+ \text{sim}_\phi(s, g^+)) + C^+ \quad (3)$$

$$\log p_-(s) = \log [C(\kappa^-) \mathbb{E}_{\mathcal{B}^-} \exp(\kappa^- \text{sim}_\phi(s, g^-))] = \log \mathbb{E}_{\mathcal{B}^-} \exp(\kappa^- \text{sim}_\phi(s, g^-)) + C^- \quad (4)$$

Let LogSumExp operator $\mathbb{L}_B^\kappa(X) := \frac{1}{\kappa} \log \mathbb{E}_B[\exp(\kappa X)]$. Substituting these refined log-densities back into Eq. 1 yields the complete, closed-form formulation of the potential-based rewards in HiRE:

$$\Phi_{\text{HiRE}}(s) := \frac{1}{\kappa^+} R^*(s) = \mathbb{L}_{\mathcal{B}^+}^{\kappa^+}(\text{sim}_\phi(s, g^+)) - \lambda \cdot \mathbb{L}_{\mathcal{B}^-}^{\kappa^-}(\text{sim}_\phi(s, g^-)) \quad (5)$$

where $\lambda = \kappa^- / \kappa^+$ naturally emerges as the ratio of the two concentration parameters that aligns the landscapes of the success and failure manifolds. The additive constant shift $C^+ - C^-$ is omitted as a state-independent constant shift in the reward function as it does not alter the optimal policy or the policy gradient direction in reinforcement learning [14]. Crucially, the magnitude of λ reflects the geometric constraints of tasks: precision-critical assembly tasks naturally demand $\lambda < 1$ ($\kappa^+ > \kappa^-$) because the successful manifold is extremely narrow and localized, i.e. high concentration, whereas failure modes are broad and generic. This configuration inherently shapes an asymmetric potential field, providing a steep, sharp attraction gradient toward the microscopic goal alongside a broad, gentle repulsion basin that steers the agent away from failure traps without suppressing local exploration.

Finally, to augment foundation representations with control awareness while preserving the optimality of the policy, we integrate this control-aware potential Φ_{HiRE} with standard task success indicators $R_{\text{sparse}}(s, a, s')$ (i.e. sparse reward) via potential-based reward shaping [14]:

$$R_{\text{HiRE}}(s, a, s') = R_{\text{sparse}}(s, a, s') + w(\gamma \Phi_{\text{HiRE}}(s') - \Phi_{\text{HiRE}}(s)) \quad (6)$$

where γ is the discount factor, ensuring that the edited reward landscape provides dense guidance without introducing any unintended optima deviated from task objective.

Intuitive interpretations. The subtractive structure of the HiRE potential in Eq. 5 directly corrects the control-unawareness of raw foundation representations, whose conventional similarity-based rewards rely strictly on passive visual matching and remain unaware of physical task dynamics. By contrasting hindsight interaction data, HiRE partitions the representation space into three distinct geometric regimes that systematically resolve the misalignment between visual similarity and control objectives: (1) **Success-dominated regime (correcting false negatives):** Raw foundation representations evaluate task progress purely upon visual features, routinely underestimating intermediate states that are dynamically correct yet appear visually distant from the final goal (e.g., a correct motion reaching the target object that occurs far from the ultimate placement site). HiRE elevates the potential of these states by aggregating successful histories, injecting a strong attractive gradient that rescues these false negatives from being under-rewarded due to visual disparity. (2) **Failure-dominated regime (correcting false positives):** Conversely, semantic similarity is inherently blind to physical constraints, making it highly prone to overestimating deceptive “trap states”—states that appear visually identical to the goal but have suffered irreversible physical failures (e.g., moving toward the placement site after dropping the small object). HiRE imposes a severe penalty on these zones, explicitly suppressing visual reward hacking that drives the robot into physical failures. (3) **Equilibrium regime (gradient cancellation):** In equilibrium zones or vast under-covered regions, the gradients of the success and failure terms achieve a dynamic counter-balance. Mathematically, even though the potential values do not identically sum to zero due to $\lambda < 1$, the reward landscape converges to a flat constant plateau where the localized shaping gradient vanishes, i.e., $\nabla_s \Phi_{\text{HiRE}}(s) \approx \mathbf{0}$. This gradient cancellation allows HiRE to gracefully degrade to the clean sparse-reward recipe. Consequently, it prevents HiRE from issuing conflicting signals on states from blurred manifolds and ensures the robot is not overly penalized by localized failures when exploring distant, out-of-distribution states.

4.2 RL Policy Finetuning with HiRE

To dynamically evaluate the non-parametric densities, HiRE maintains a positive buffer $\mathcal{B}^+$ and a negative buffer $\mathcal{B}^-$ during policy finetuning. The positive buffer $\mathcal{B}^+$ consists of an offline buffer constructed from expert demonstrations, together with an online first-in-first-out (FIFO) buffer that stores observations from recent successful rollouts. The negative buffer $\mathcal{B}^-$ is another FIFO buffer, populated only with terminal states from failed rollouts to illustrate the failure manifold. As HiRE only reshapes the intrinsic reward signal, it is agnostic to the underlying RL algorithm. We formalize this plug-and-play integration through a generalized actor-critic objective. Let $\mathcal{U}$ denote the sampling distribution from which the backbone draws training transitions (s, a, s') . The critic Q_ψ is updated by minimizing the shaped Bellman residual:

$$\mathcal{L}_\psi = \mathbb{E}_{(s,a,s') \sim \mathcal{U}} [Q_\psi(s, a) - (R_{\text{HiRE}}(s, a, s') + \gamma \mathbb{E}_{a' \sim \pi_\theta} [Q_{\bar{\psi}}(s', a')])]^2 \quad (7)$$

where $Q_{\bar{\psi}}$ is the target network; $\bar{\psi}$ is periodically copied from ψ ; $R_{\text{HiRE}}(s, a, s')$ in Eq. 6 represents our dense reward recipe that draw policies toward success and away from failure, where $\lambda \in [0, 1]$ as success manifolds are usually higher concentrated than failure ones. Subsequently, the actor π_θ is optimized to minimize the negative estimated shaped return Q_ψ alongside a flexible regularization term $\mathcal{L}_{\text{algo}}$ which depends on choice of RL algorithm `algo`:

$$\mathcal{L}_\theta = \mathbb{E}_{s \sim \mathcal{U}, a \sim \pi_\theta} [-\omega_{\text{algo}}(s, a) \cdot Q_\psi(s, a) + \mathcal{L}_{\text{algo}}] \quad (8)$$

This abstraction elegantly accommodates various RL backbones: setting $\omega_{\text{algo}} = 1$ recovers standard deterministic or stochastic off-policy methods; incorporating ω_{algo} with clipped importance sampling constraints yields stable on-policy updates (e.g., PPO [27]); and applying behavior regularization $\mathcal{L}_{\text{algo}}$ enables entropy-based exploration bonus for stochastic policies (e.g., TD3 [37], SAC [38]) or safe finetuning from the base policy (e.g., DICE-RL [15]). In our main implementations, we integrate our reward recipe on top of DICE-RL, a stable and sample-efficient RL policy finetuning algorithm.

5 Experiments

We evaluate HiRE in the setting of online RL finetuning of pretrained robot policies. We adopt DINOv2 [10] and SigLIP [11] as foundation encoders, and DICE-RL [15], a stable and sample-efficient finetuning framework for generative behavior cloning policies, as policy learning backbone.

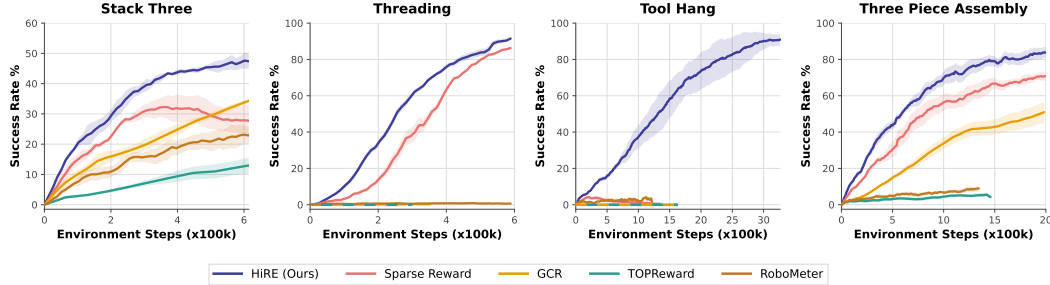


Figure 1: **Comparisons on RoboMimic and MimicGen tasks.** Success rate versus online environment steps for different reward recipes integrated with DICE-RL policy learning backbone. Curves are averaged over three random seeds, with shaded regions denoting variability across seeds.

Building on such a competitive backbone lets us cleanly isolate the contribution of the reward signal, and ask whether a better reward can push an already strong finetuning pipeline even further. Concretely, our experiments are designed around three questions: Can HiRE improve an RL finetuning backbone with dense and correct guidance? How does HiRE compare with learned reward adaptation, pretrained robotic reward models, and training-free VLM rewards? How does HiRE dynamically shape reward landscape that eventually lifts policy finetuning ceiling? Implementation details, additional comparative, hyperparameter sensitivity and ablation results are elaborated on in App. C.

5.1 Experimental Setup

(1) Benchmarks and tasks. We evaluate HiRE on both simulation and real-robot manipulation tasks. We use RoboMimic [17] and MimicGen [16] simulation benchmarks, and test on two different manipulation tasks with I2RT YAM robot arms for real-robot experiments, detailed in App. A.1 and A.2 respectively. Together, these tasks span prehensile, long-horizon, contact-rich, and high-precision manipulation, with several requiring high-precision insertion and assembly. For each task, we first train generative behavior cloning (BC) policies from offline demonstrations on two benchmarks respectively, achieving base policies with success rates of 18.0% on *Stack_Three*, 6.0% on *Three_Piece_Assembly*, 1.0% on *Threading*, 2.0% on *Tool_Hang*, and 20.0% on both hardware tasks, i.e., *Bottle_Pick-and-Place* and *Towers_of_Hanoi*, and then finetune them with DICE-RL. **(2) Reward baselines.** We compare HiRE with four reward baselines. **Sparse reward** uses binary task success signals. It is easy to obtain in both simulation and real-world settings but provides no intermediate feedback before task completion, which falls short on long-horizon multi-stage tasks. **RoboMeter** [4] (with RoboMeter-4B configuration) is a general-purpose robotic dense reward model trained with frame-level progress supervision and trajectory-level preference supervision. It learns from trajectory comparisons among expert, suboptimal, and failed rollouts, with large-scale task-specific human labeling. **TOPReward** [5] (with Qwen3-VL-8B-Instruct configuration) is a training-free robotic reward method that estimates task progress from a pretrained VLM. It scores task-completion token probabilities conditioned on trajectory prefixes and task instructions, without task-specific reward-model training or human progress annotations. **GCR** [12] learns dense rewards by combining implicit value learning with a goal-contrastive objective that distinguishes successful and failed trajectories. It provides a learned reward-adaptation baseline for comparison with HiRE’s training-free reward editing.

5.2 Benchmark Comparative Results

Long-horizon, multi-stage tasks (*Stack_Three*, *Three_Piece_Assembly*). For these long-horizon tasks, **HiRE lifts the final success rate substantially above the sparse reward baseline** as shown in Fig. 1. On *Stack_Three*, sparse-reward policy finetuning initially improves but later declines to around 30%, while HiRE continues to improve and reaches a success rate of roughly 50%. On *Three_Piece_Assembly*, sparse-reward finetuning reaches a final success rate of around 70%, while HiRE climbs to over 80%. Since both tasks require several ordered sub-stages before terminal success, sparse rewards provide limited credit assignment within available interaction budgets. In contrast, HiRE supplies dense progress feedback, leading to higher performance.

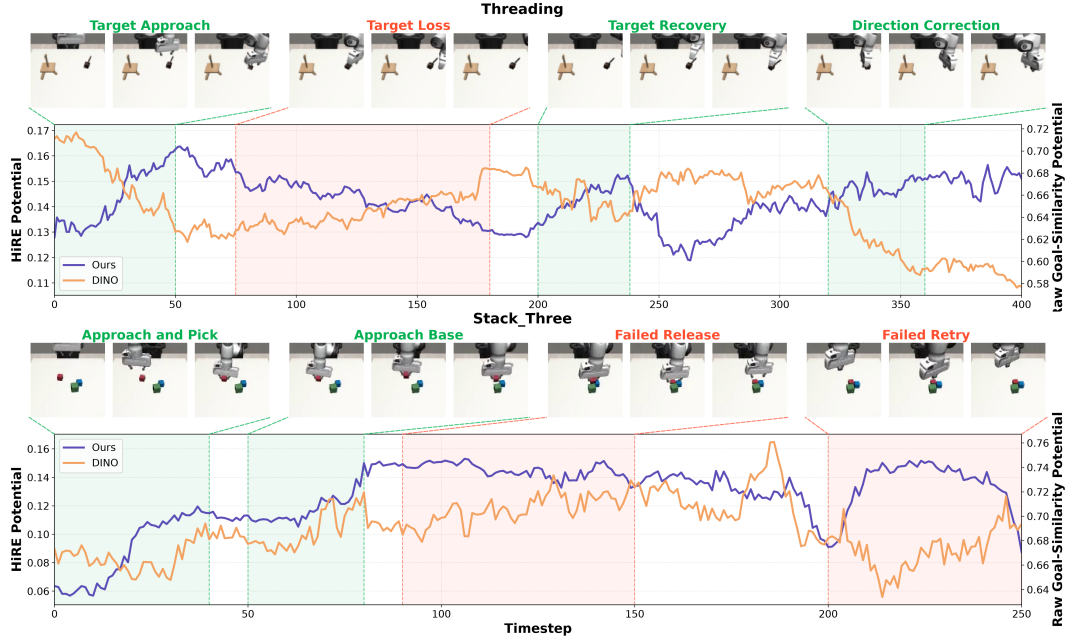


Figure 2: **HiRE reflects task progress better than naïve visual similarity-based rewards.** We visualize reward scores along two representative trajectories, with selected segments highlighted and shown as thumbnails above the plot. The shaded regions correspond to key behavioral phases, including target approach, target loss, target recovery, and direction correction. While DINO similarity-based potentials mainly reflect visual resemblance to the ultimate goal frame, the HiRE potential remains faithful to reflect task progress and penalizes visually plausible but failure-prone states.

Contact-rich and high-precision tasks (*Tool_Hang*, *Threading*). For contact-rich and precision tasks, HiRE’s advantage appears in two related but distinct aspects. On *Tool_Hang*, sparse-reward finetuning fails completely despite non-zero base policy performance, whereas HiRE steadily improves to a success rate of around 95% as shown in Fig. 1. HiRE provides dense feedback on intermediate progress, helping the policy learn useful manipulation behaviors even when task successes are rare. On *Threading*, though the sparse-reward recipe eventually reaches a comparable final success rate, **HiRE always reaches the same performance with markedly fewer online interactions**, showing clear gains in sample efficiency.

Comparison with other reward baselines. The above results show that HiRE raises performance when sparse rewards fail to capture intermediate progress and at least improves sample efficiency when sparse supervision is eventually sufficient. We further compare HiRE with pretrained reward models and methods that adapt rewards online. The pretrained robotic reward model **RoboMeter** and the VLM-based reward method **TOPReward** exhibit inconsistent performance across tasks and achieve substantially lower final success rates than HiRE, suggesting limited generalization of their reward signals to these unseen tasks and scenarios. **GCR** also adapts rewards using online failure experience, but requires gradient-based reward-model updates. On the multi-stage tasks (*Stack_Three*, *Three_Piece_Assembly*), GCR requires roughly 3 times as many interactions as HiRE to reach 30% success. On the high-precision tasks (*Tool_Hang*, *Threading*), it fails to make substantial progress. Together, these comparisons highlight HiRE’s sample efficiency and performance consistency across tasks, achieved through its training-free dense reward recipe.

5.3 Real-Robot Comparative Results

We evaluate each method on two real-robot tasks, comparing their success rates per checkpoint in Fig. 3. On *Bottle_Pick-and-Place*, HiRE steadily guides the policy toward better performance, boosting the success rate from 20% to 67%, whereas the sparse-reward and Robometer baselines nearly fail entirely in the early training phases. Detailed performance across all six initial positions is further reported in Tab. 2. On *Towers_of_Hanoi*, HiRE instantly boosts the success

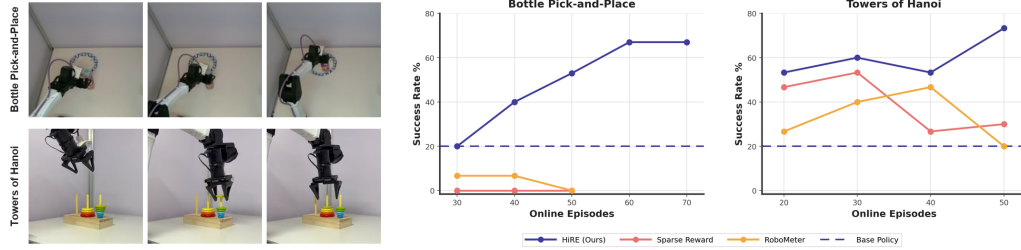


Figure 3: **Real-robot evaluation.** **Left:** Representative successful HiRE rollouts for Bottle_Pick-and-Place (top) and Towers_of_Hanoi (bottom). **Right:** Success rates over online fine-tuning episodes. Starting from the same base policy (20%), HiRE reaches 67% on Bottle_Pick-and-Place and 73% on Towers_of_Hanoi, consistently outperforming sparse-reward and RoboMeter baselines.

rate to 53.3% with only 20 episodes, eventually reaching a peak of **73.3%** ($3.67\times$ base policy). In contrast, the baselines achieve substantially lower peak performances (Sparse: 53.3% at 30 episodes; Robometer: 46.7% at 40 episodes), requiring more interactions to reach inferior caps. While HiRE maintains steady performance gains, both baselines suffer from severe degradation during fine-tuning (e.g., Sparse drops to 30.0% and Robometer to 20.0% at 50 episodes) due to reward hacking or noisy reward estimates. In conclusion, HiRE achieves superior sample efficiency, stable improvement, and higher policy performance ceiling on both hardware tasks.

This significant contrast stems from HiRE’s ability to extract dense optimality signals from the positive and negative buffers, which efficiently activates and steers the base policy toward successful cases. Conversely, sparse rewards fail to provide sufficient optimization gradients. As illustrated by the critic loss comparison during online training on Bottle_Pick-and-Place in Fig. 4, sparse rewards yield a prematurely low critic loss due to the severe sparsity of reward signals, leading to value function collapse. In contrast, Robometer induces a non-converged critic loss, probably because it introduces extrinsic rewards that are not robust with OOD observations. HiRE maintains non-trivial critic updates throughout the finetuning process, demonstrating that its edited potential field continuously delivers informative feedback to sustain effective policy learning.

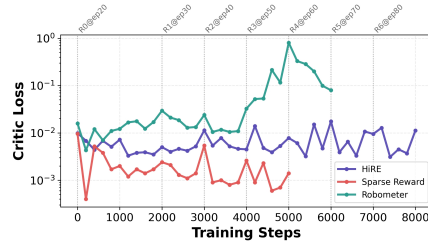


Figure 4: **Critic loss.** We compare critic loss between HiRE and baselines during policy finetuning. HiRE maintains non-trivial critic updates, while sparse reward yields significantly lower critic loss due to its sparse signals, and Robometer probably produces inconsistent rewards that are hard for the critic to converge.

5.4 Visualizations and Analyses

HiRE endows foundation vision representations with control-awareness. In Fig. 2, we compare the raw DINO goal-similarity potential $\Phi_{\text{raw}}(s) = \text{sim}_{\phi}(s, g_{\star}^+)$ commonly adopted by existing representation-based reward learning methods [8, 9, 22], against our edited potential $\Phi_{\text{HiRE}}(s)$ in Eq. 5 on Threading and Stack_Three trajectories. Taking Threading subplot as an example, the robot correctly approaches the target object during the early execution phase (e.g. Timestep 0-50). However, because this leads to observations visually distinct from the ultimate goal frame $g_{\star}^+$, $\Phi_{\text{raw}}(s)$ severely degrades, potentially causing optimization misdirection. In sharp contrast, $\Phi_{\text{HiRE}}(s)$ successfully aggregates historical success data to steadily elevate the reward, reinforcing this critical task progress and correcting such false negatives of $\Phi_{\text{raw}}(s)$ caused by visual disparity. Conversely, a more catastrophic visual artifact occurs during the bad navigation phases (e.g. Timestep 75-180), where the robot loses its target and tracks overshoots, causing its leaving the camera view. Paradoxically, $\Phi_{\text{raw}}(s)$ exhibits a deceptive increase, falsely interpreting the empty background as a closer match to $g_{\star}^+$. These highlight a fundamental flaw in raw rewards derived from foundation vision representations: $\Phi_{\text{raw}}(s)$ is inherently vulnerable to both *false negatives* (under-rewarding

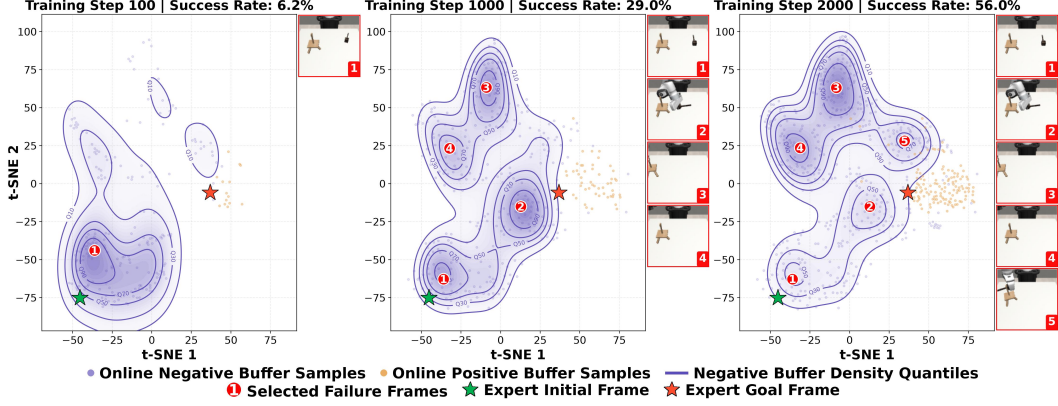


Figure 5: **The negative buffer captures distinct failure behaviors during policy finetuning.** We visualize positive and negative buffer samples in a shared t-SNE space across training steps. Online positive buffer samples correspond to successful rollouts whereas negative ones are the terminal frames of failure rollouts. Density quantile contours highlight high-density regions of the negative-buffer distribution, where selected failure frames are sampled as representative examples. Expert initial and goal frames are plotted as reference markers.

visually incorrect yet beneficial behaviors) and *false positives* (over-rewarding visually correct yet improper behaviors). By attracting towards success and repelling from failure manifolds, HiRE systematically unmask these visual illusions and grounds control-awareness into rewards.

HiRE co-adapts the reward manifold upon policy finetuning performance. In Fig. 5, we analyze the joint t-SNE embedding space of both online positive and negative buffer samples across training steps. The empirical density quantile contours reveal two profound insights into dynamical reward shaping nature of HiRE. First, online negative buffer dynamically captures stage-progressive failure modes while preventing over-penalization. At the beginning, the negative samples concentrates around the initial state (Region 1). As the policy performance improves, the buffer removes these early, trivial failures and segments into multiple localized clusters (Regions 2-5) that encapsulate harder, later-stage failures. HiRE thus avoids global over-penalization and maintains a targeted shaping to only suppress localized failures along training. Second, the positive distribution remains highly concentrated whereas the negative one is much wider and scattered. This empirical discrepancy confirms assigning a larger concentration factor κ^+ than to κ^- , which we discuss in Sec. 4.1.

6 Conclusion and Limitations

HiRE addresses the reward bottleneck in RL policy finetuning by endowing foundation-representation similarity with control-awareness. Without training reward models or using human labels, HiRE edits rewards online through hindsight evidence from successful and failed rollouts, suppressing deceptive trap states while preserving genuine task progress. Across RoboMimic, MimicGen, and real-robot platforms, this lightweight calibration consistently improves both final success rate and sample efficiency over other reward methods, while remaining agnostic to the encoder and finetuning algorithm. Overall, HiRE offers a practical and scalable recipe that helps existing RL finetuning pipelines unlock their full potential. **Limitations and Future Work.** While success and failure signals offer highly accessible and clean feedback, they overlook intermediate, control-centric information. Future extensions could incorporate human guidance to make interaction data more control-aware, thereby shaping a more nuanced reward landscape. Another promising direction is to develop more sophisticated buffer construction strategies that account for suboptimal segments within successful trajectories, such as segments involving retries or recovery behaviors of varying degrees.

Acknowledgement

We express our sincere gratitude to Neural Motion and the Tianqiao & Chrissy Chen Institute for providing essential computational resources for this work. In particular, we deeply appreciate Neural Motion for supplying critical GPU compute in the final month prior to submission. We also thank

Zhanyi Sun and Qianzhong Chen from Stanford University for their valuable insights and technical guidance on implementation details.

References

- [1] P. Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, et al. $\pi_{0.6}^*$: a vla that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025. 1
- [2] C. Xu, J. T. Springenberg, M. Equi, A. Amin, A. Esmail, S. Levine, and L. Ke. Rl token: Bootstrapping online rl with vision-language-action models. *arXiv preprint arXiv:2604.23073*, 2026. 1, 3
- [3] T. Lee, A. Wagenmaker, K. Pertsch, P. Liang, S. Levine, and C. Finn. Roboreward: General-purpose vision-language reward models for robotics. *arXiv preprint arXiv:2601.00675*, 2026. 1, 2
- [4] A. Liang, Y. Korkmaz, J. Zhang, M. Hwang, A. Anwar, S. Kaushik, A. Shah, A. S. Huang, L. Zettlemoyer, D. Fox, Y. Xiang, A. Li, A. Bobu, A. Gupta, S. Tu, E. Biyik, and J. Zhang. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. In *Robotics: Science and Systems 2026*, 2026. 1, 2, 6, 16
- [5] S. Chen, C. Harrison, Y.-C. Lee, A. J. Yang, Z. Ren, L. J. Ratliff, J. Duan, D. Fox, and R. Krishna. Topreward: Token probabilities as hidden zero-shot rewards for robotics. *arXiv preprint arXiv:2602.19313*, 2026. 1, 3, 6, 17
- [6] H. Tan, S. Chen, Y. Xu, Z. Wang, Y. Ji, C. Chi, Y. Lyu, Z. Zhao, X. Chen, P. Co, et al. Robo-dopamine: General process reward modeling for high-precision robotic manipulation. *arXiv preprint arXiv:2512.23703*, 2025. 1, 2
- [7] J. Rocamonde, V. Montesinos, E. Nava, E. Perez, and D. Lindner. Vision-language models are zero-shot reward models for reinforcement learning. In *International Conference on Learning Representations*, volume 2024, pages 28446–28463, 2024. 1, 3
- [8] Y. J. Ma, S. Sodhani, D. Jayaraman, O. Bastani, V. Kumar, and A. Zhang. Vip: Towards universal visual reward and representation via value-implicit pre-training. *arXiv preprint arXiv:2210.00030*, 2022. 1, 2, 8
- [9] Y. J. Ma, V. Kumar, A. Zhang, O. Bastani, and D. Jayaraman. Liv: Language-image representations and rewards for robotic control. In *International Conference on Machine Learning*, pages 23301–23320. PMLR, 2023. 1, 2, 8, 17
- [10] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. HAZIZA, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=a68SUt6zFt>. Featured Certification. 1, 2, 3, 5, 15
- [11] M. Tschannen, A. Gritsenko, X. Wang, M. F. Naeem, I. Alabdulmohsin, N. Parthasarathy, T. Evans, L. Beyer, Y. Xia, B. Mustafa, et al. Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv preprint arXiv:2502.14786*, 2025. 1, 2, 5, 18
- [12] O. Biza, T. Weng, L. Sun, K. Schmeckpeper, T. Kelestemur, Y. J. Ma, R. Platt, J.-W. van de Meent, and L. L. Wong. On-robot reinforcement learning with goal-contrastive rewards. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4797–4805. IEEE, 2025. 2, 6, 17

- [13] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne. Experience replay for continual learning. *Advances in neural information processing systems*, 32, 2019. 2
- [14] A. Y. Ng, D. Harada, and S. J. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 278–287, 1999. 2, 4
- [15] Z. Sun and S. Song. From prior to pro: Efficient skill mastery via distribution contractive rl finetuning. *arXiv preprint arXiv:2603.10263*, 2026. 2, 3, 5, 14
- [16] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. In *Conference on Robot Learning*, pages 1820–1864. PMLR, 2023. 2, 6, 13
- [17] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *Conference on Robot Learning*, pages 1678–1690. PMLR, 2022. 2, 6, 13
- [18] T. Mu, M. Liu, and H. Su. Drs: Learning reusable dense rewards for multi-stage tasks. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=6CZ50WgfCG>. 2
- [19] Q. Chen, J. Yu, M. Schwager, P. Abbeel, F. Shentu, and P. Wu. Sarm: Stage-aware reward modeling for long horizon robot manipulation. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=aemqAxSc19>. 2
- [20] D. Yang, D. Tjia, J. Berg, D. Damen, P. Agrawal, and A. Gupta. Rank2reward: Learning shaped reward functions from passive video. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2806–2813. IEEE, 2024. 2
- [21] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta. R3m: A universal visual representation for robot manipulation. In *Conference on Robot Learning*, pages 892–909. PMLR, 2023. 2
- [22] J. Li, J. Zheng, Y. Zheng, L. Mao, X. Hu, S. Cheng, H. Niu, J. Liu, Y. Liu, J. Liu, et al. Decisionnce: embodied multimodal representations via implicit preference learning. In *Proceedings of the 41st International Conference on Machine Learning*, pages 29461–29488, 2024. 2, 8
- [23] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021. 3
- [24] P. Krack, T. Jülg, W. Burgard, and F. Walter. Rewarding dino: Predicting dense rewards with vision foundation models. *arXiv preprint arXiv:2603.16978*, 2026. 3
- [25] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 3
- [26] K. Chen, Z. Liu, T. Zhang, Z. Guo, S. Xu, H. Lin, H. Zang, Q. Zhang, Z. Yu, G. Fan, et al. π_{RL} : Online rl fine-tuning for flow-based vision-language-action models. *arXiv preprint arXiv:2510.25889*, 2025. 3
- [27] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 3, 5
- [28] A. Wagenmaker, Y. Zhang, M. Nakamoto, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine. Steering your diffusion policy with latent space reinforcement learning. In *Conference on Robot Learning*, pages 258–282. PMLR, 2025. 3

- [29] L. Ankile, Z. Jiang, R. Duan, G. Shi, P. Abbeel, and A. Nagabandi. Residual off-policy rl for finetuning behavior cloning policies. *arXiv preprint arXiv:2509.19301*, 2025. 3
- [30] X. Yuan, T. Mu, S. Tao, Y. Fang, M. Zhang, and H. Su. Policy decorator: Model-agnostic online refinement for large policy model. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=e5jGTEiJMT>. 3
- [31] M. Nakamoto, O. Mees, A. Kumar, and S. Levine. Steering your generalists: Improving robotic foundation models via value guidance. In P. Agrawal, O. Kroemer, and W. Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 4996–5013. PMLR, 06–09 Nov 2025. URL <https://proceedings.mlr.press/v270/nakamoto25a.html>. 3
- [32] R. S. Sutton, A. G. Barto, et al. Introduction to reinforcement learning. 1998. 3
- [33] J. Ho and S. Ermon. Generative adversarial imitation learning. *Advances in neural information processing systems*, 29, 2016. 3
- [34] J. Fu, K. Luo, and S. Levine. Learning robust rewards with adversarial inverse reinforcement learning. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rkHywl-A->. 3
- [35] B. D. Ziebart, A. L. Maas, J. A. Bagnell, A. K. Dey, et al. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pages 1433–1438. Chicago, IL, USA, 2008. 4
- [36] K. V. Mardia and P. E. Jupp. *Directional statistics*. John Wiley & Sons, 2009. 4
- [37] S. Fujimoto, H. Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, pages 1587–1596, 2018. 5
- [38] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pages 1861–1870, 2018. 5
- [39] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, S. Nasiriany, and Y. Zhu. robo-suite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 13
- [40] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023. 13
- [41] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021. 16
- [42] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine. Efficient online reinforcement learning with offline data. *arXiv preprint arXiv:2302.02948*, 2023. 16
- [43] S. Boyd, S. P. Boyd, and L. Vandenberghe. *Convex optimization*. Cambridge university press, 2004. 17

Appendix

A Implementation Details

This section provides the implementation details for the main experimental comparison in Fig. 1. We first describe the common setup, and then detail the reward implementation for HiRE and all the baselines in Section A.3.

A.1 Benchmark and Policy Setup

Benchmarks. We use the standard RoboMimic [17] and MimicGen [16] simulation environments built on Robosuite v1.4.1 [39]. The main experiments include `Tool_Hang` from RoboMimic, `Threading`, `Stack_Three`, and `Three_Piece_Assembly` from MimicGen. All environments use an `OSC_POSE` controller at 20 Hz and a 7-D action space, consisting of 6-DoF end-effector pose deltas and a gripper command. Task completion is evaluated with the environment-provided binary success signal. Since the simulation environments provide no failure detection mechanism, an episode is labeled as failed only if it reaches a sufficiently long timeout horizon without completing the task. Reaching this limit implies the robot either failed outright or got stuck in a local trap state, both of which are undesirable behaviors that warrant penalization. Storing only the final failure frame in $\mathcal{B}^-$ is a concise and effective implementation that performs well in our experiments as in Fig. 1. This design can also be extended to retain the last few frames of failed episodes when buffer capacity and computation permit. However, including the entire failed trajectory would mistakenly penalize valid intermediate states that also appear in successful data, which would unnecessarily suppress good behaviors.

Observation. The policy observation contains two RGB camera (wrist-view and base-view) and 9-D proprioception (end-effector position, quaternion, and gripper state). The trainable policy encoder fuses the two camera views into a 128-D visual embedding, which is concatenated with proprioception to form the state input for the residual actor and critic during RL finetuning.

Behavior-cloning pretraining. Before online finetuning, we train one behavior-cloning policy per task from the corresponding offline demonstrations. Each policy is optimized with Adam (learning rate 10^{-4} , weight decay 10^{-6}) and a cosine schedule. We maintain an exponential moving average of the weights with decay 0.995 and use the EMA checkpoint as the initial policy for all reward baselines. Thus sparse reward, RoboMeter, and HiRE start from exactly the same pretrained policy in each task and seed.

Policy architecture. All baselines share the end-to-end trainable policy architecture summarized in Table 1: a ResNet-18 with spatial softmax for visual encoding and a 1D UNet action head. The MimicGen tasks use a diffusion policy [40] with ϵ -prediction with 50 DDPM steps; RoboMimic `Tool_Hang` uses a flow-matching policy with velocity prediction and 20-step Euler integration.

Table 1: Policy architecture.

Component	Value
Visual encoder	ResNet-18 + spatial-softmax, trained end-to-end
Keypoints / fusion	32 keypoints/cam, 64-D/cam $\rightarrow$ fused 128-D
Action head	1D UNet, base 128, multipliers [1, 2, 2]
Horizons	$T_a = 8$, $T_o = 1$, $T_{img} = 1$

A.2 Real-Robot Implementations, Setups, and Results

Hardware setup. Our real-robot experiments are conducted on the I2RT YAM arm, a 6-DoF manipulator equipped with a parallel linear4310 gripper. We adopt the teleoperation workstation configuration, which ships with a leader-follower system that allows a human operator to collect expert demonstrations through teleoperation. Visual observations are captured by two Intel RealSense cameras, with a base-view camera providing a global top view of the workspace (base camera) and a wrist-view camera mounted on the end-effector providing a close-up egocentric view of the gripper

and manipulated objects (wrist camera), as shown in Fig. 7 that use `Bottle_Pick-and-Place` as an example. On real hardware, successes and failures are directly judged by human operators.

Task descriptions. `Bottle_Pick-and-Place` requires the robot to localize a bottle, grasp it reliably, transport it to a target plate, and release it accurately. The complexities of this task are two-fold: (1) **Coarse-to-fine compliance and precision:** The task demands a seamless integration of large-scale macro-movements (e.g., reaching for the bottle or transporting it toward the plate) and high-precision micro-manipulation (e.g., at the moments of grasping and placing). Crucially, because both the plastic bottle and the paper plate are highly deformable, the policy must maintain compliant control during grasping and releasing to prevent the objects from bouncing away or slipping. (2) **Spatial generalization:** We evaluate the policies across six different bottle initial positions spanning a wide tabletop workspace (as shown in Fig. 6), testing the robust generalization of HiRE rewards. We randomize the initial bottle positions within a small range approximately of $\pm 10\text{cm}$ on the table. We elaborate on the performance of HiRE on each initial position in Tab. 2.

`Towers_of_Hanoi` requires the robot to move the green disk from the rightmost rod to the middle rod of a Hanoi platform, demanding a sequence of high-precision manipulations. Although the initial setup is standardized across trials for reproducibility, the Hanoi base is not anchored to the tabletop. Consequently, execution uncertainties often cause the main platform to shift relative to the workspace, while the removable top cover can independently slide relative to the base. These unconstrained spatial shifts introduce severe dynamic perturbations and non-recoverable error states, where even a minor contact alignment error can render the task physically unrecoverable, imposing strict demands on real-time policy precision and feedback robustness.

Policy training and evaluation. We collect diverse 117 teleoperated expert demonstrations for `Bottle_Pick-and-Place` (the distribution density is visualized in Fig. 7 with colors) and 50 for `Towers_of_Hanoi` to initialize base diffusion policies, following the exact implementation recipe of DICE-RL [15]. For online finetuning, the pre-trained diffusion policy first executes 20 warmup episodes distributed evenly across the designated initial positions. We then conduct policy optimization for 2,000 training steps with a batch size of 256. To maintain a stable update-to-data (UTD) ratio during interaction, the policy samples 10 new episodes after every 1,000 training steps in subsequent training loops, ensuring a consistent training stability and data exploitation.

We evaluate HiRE, sparse-reward and RoboMeter baselines on the YAM robot arms under the same setup. All methods start from the same pre-trained policy and 20 warmup episodes; the only difference is the reward signal used during finetuning. For all methods, a human annotator labels each episode as success or failure during execution. For HiRE, we set positive buffer size to 4096 and negative buffer size to 320. We add all the online successful episodes into the positive buffer. We add the last 16 frames of online failed episodes into the negative buffer. We set $\kappa^+ = 10$, $\kappa^- = 1$, $\lambda = 0.1$, and batch size 256. All real-robot experiments are conducted on a single NVIDIA RTX 5090 GPU. In Fig. 3, the reported success rates are evaluated across 30/15 trials for each checkpoint on `Bottle_Pick-and-Place/Towers_of_Hanoi`.

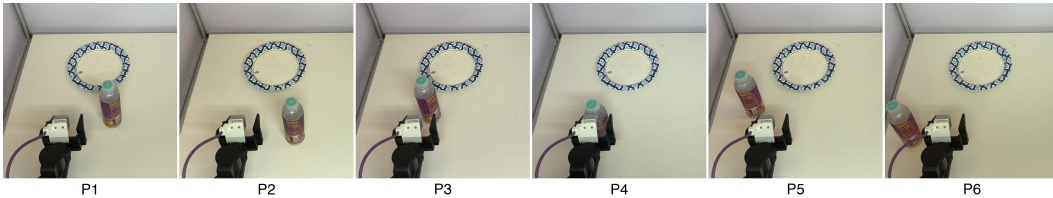


Figure 6: **Six fixed initial bottle positions (P1–P6) used for real-robot evaluation.** Each panel shows the base-view camera observation at the start of an episode, with the bottle placed at one of the six predefined table positions.

A.3 Method Implementations

The main comparison in Fig. 1 evaluates HiRE against four reward baselines: sparse reward, RoboMeter, TOPReward, and GCR. All reward methods are integrated into the same DICE-RL backbone [15];

Table 2: **Detailed Real-robot evaluation rollout success rates (HiRE) on Bottle-Pick-and-Place.** Each policy is evaluated at six fixed initial positions (P1–P6) with five trials per position, for 30 trials in total. Each cell reports the number of successful attempts out of five, and the last two columns report the overall success count and success rate. BC denoted the pre-trained base policy; the remaining rows are checkpoints taken at increasing numbers of online finetuning episodes.

Policy	P1	P2	P3	P4	P5	P6	Success	Rate
BC	2	0	0	2	0	2	6/30	20.0%
Online ep. 30	1	2	1	0	1	1	6/30	20.0%
Online ep. 40	4	4	2	1	1	0	12/30	40.0%
Online ep. 50	3	5	2	1	2	3	16/30	53.3%
Online ep. 60	3	5	3	3	3	3	20/30	66.7%
Online ep. 70	4	3	3	4	3	3	20/30	66.7%

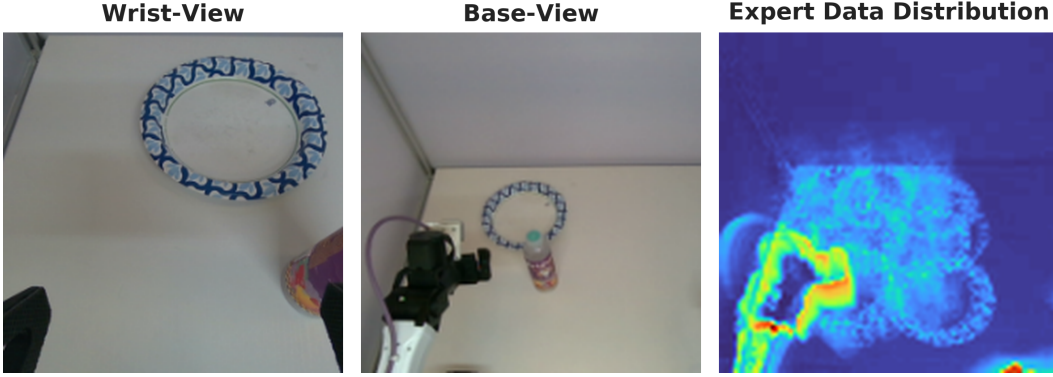


Figure 7: **Wrist view, base view, and visualization of offline expert data.**

only the reward computation differs across methods. DICE-RL freezes the pretrained generative policy as an action proposal model and learns a residual actor together with an ensemble critic. We keep all optimization, replay, exploration, and evaluation settings fixed across reward baselines; only the scalar reward stored for each transition changes. Table 3 lists the hyperparameters. For visibility, curves with zero success throughout their recorded evaluations are extended at 0% to the midpoint of the displayed training range.

HiRE reward. HiRE keeps the same sparse success signal as the sparse baseline and augments it with the control-aware potential Φ_{HiRE} of Eq. 5, edited online from hindsight rollouts. The potential is built on a frozen DINOv2 [10] (dinov2_vits14, ViT-S/14) encoder ϕ , which we evaluate inside the main training process on both camera views. Each image is resized to 224×224 , normalized with ImageNet statistics, and encoded into P normalized patch tokens. The state–reference similarity sim_ϕ in Eq. 5 is the patch-averaged cosine similarity which is averaged across the two cameras.

$$\text{sim}_\phi(s, g) = \frac{1}{P} \sum_{p=1}^P \frac{\phi_p(s)^\top \phi_p(g)}{\|\phi_p(s)\| \|\phi_p(g)\|}, \quad (9)$$

The positive buffer $\mathcal{B}^+$ and negative buffer $\mathcal{B}^-$ of Eq. 5 are instantiated as follows. The positive buffer $\mathcal{B}^+$ combines an offline component, constructed once from the expert demonstrations with frame stride 5, and an online FIFO component of capacity 128 that stores terminal frames from online successful rollouts; the two components are mixed with ratio 0.5 when sampling. The negative buffer $\mathcal{B}^-$ is an online FIFO buffer of capacity 128, populated only with the terminal frames of failed rollouts. For online positives, retaining only the successful terminal frame avoids reinforcing exploratory or suboptimal intermediate behaviors, while offline expert demonstrations provide positive references

Table 3: DICE-RL finetuning hyperparameters.

Setting	Value
Residual actor	3-layer MLP, hidden 1 024, GELU, LayerNorm; 56-D noise
Action decoding	DDIM [41], 10 steps
Critic	ensemble of 10; 16 candidates, <code>max_q_min</code> selection
Collection	100 steps/iter $\times$ 8 parallel envs
Updates per iter	20, batch 256
Actor : critic updates	1 : 2; target soft update $\tau = 0.01$
Optimizer	Adam, lr 10^{-4} , weight decay 10^{-5} , cosine warm-up
Returns	5-step, $\gamma = 0.99$
Replay buffer	up to 2×10^6 transitions
Evaluation	every 200 steps, 10 episodes $\times$ 10 envs

along the task trajectory. For online negatives, retaining only the failed terminal frame avoids penalizing valid intermediate behaviors that may precede failure.

Finally, Φ_{HiRE} is integrated with the sparse reward through the potential-based reward shaping (PBRs) form of Eq. 6:

$$R_{\text{HiRE}}(s, a, s') = R_{\text{sparse}}(s, a, s') + \frac{w_{\text{dense}}(t)}{T_a} (\gamma \Phi_{\text{HiRE}}(s') - \Phi_{\text{HiRE}}(s)). \quad (10)$$

We use shaping discount $\gamma = 0.99$ and normalize the shaping term by the action-chunk length $T_a = 8$. To keep the dense signal from interfering with an already competent policy, we down-weight the shaping term by a sliding-window weight $w_{\text{dense}}(t) = 1 - \text{SR}_{\text{recent}}(t)$, where $\text{SR}_{\text{recent}}(t)$ is the success rate over the most recent completed episodes. As the success rate rises, w_{dense} decays toward 0, so HiRE shapes the reward most strongly early in finetuning and gradually hands control back to the sparse task signal. Following RLPD [42], replayed expert demonstrations keep the sparse environment reward only and are not relabeled with the online contrastive reward, which keeps the learning signal clean.

Sparse reward. The sparse baseline uses the native binary success signal from the benchmark:

$$R_{\text{sparse}}(s, a, s') = \mathbb{1}\{s' \text{ is successful}\}. \quad (11)$$

All non-terminal and failed terminal transitions receive $R = 0$, and a successful terminal transition receives $R = 1$.

RoboMeter. The RoboMeter [4] baseline uses the publicly released `RoboMeter-4B` checkpoint as a frozen vision-language dense reward model. Following the official LIBERO-style inference wrapper, we run RoboMeter as a separate GPU service and query it from the RL trainer through batched HTTP requests. This decouples VLM inference from policy optimization and keeps the reward model fixed throughout finetuning. For each parallel environment, we maintain an online RGB frame buffer from base-view camera for reward inference. The episode prefix is evenly subsampled along time to at most 16 frames, always retaining the first and most recent frames. The resulting clip is conditioned on the corresponding natural-language task instruction.

The reward signal is obtained from RoboMeter’s progress head. Given the progress curve $\hat{p}(s')$ predicted from the episode prefix ending at the next state s' , we take its final prediction and clip it to $[0, 1]$:

$$p(s') = \text{clip}(\hat{p}(s')[-1], 0, 1), \quad (12)$$

Throughout this paper, RoboMeter denotes the relative-progress reward, which adds the change in predicted progress to the sparse success signal:

$$R_{\text{RM}}(s, a, s') = R_{\text{sparse}}(s, a, s') + (p(s') - p(s)). \quad (13)$$

Here $p(s)$ and $p(s')$ denote the cached progress estimates before and after the transition, respectively. We reset the progress estimate to zero at the initial state of every episode. We query RoboMeter every $N = 4$ action chunks and cache the most recent progress estimate between queries. This reward therefore measures progress increments rather than repeatedly rewarding the same absolute progress level.

TOPReward [5]. The TOPReward baseline uses the frozen Qwen3-VL-8B-Instruct model and follows the official scoring implementation. At each action chunk, we uniformly sample at most 16 base-view frames from the current episode prefix, preserving their temporal order, and condition the resulting clip on the natural-language task instruction.

The model scores the log-probability of an affirmative task-completion judgment without first generating a trajectory description. Let $\ell(s)$ denote the raw score for the episode prefix ending at state s , computed with mean token reduction. We convert successive scores into a clipped difference reward:

$$R_{\text{TOPReward}}(s, a, s') = \text{clip}(2(\ell(s') - \ell(s)), -1, 1). \quad (14)$$

The first query of each episode receives zero reward. This signal replaces the sparse environment reward for online rollouts. We apply the difference directly to raw scores, without min-max normalization across trajectory prefixes.

GCR [12]. We implement the simple-contrastive variant GCR (SC) [12] following the method and implementation details in the original paper and its appendix, as no official implementation was publicly available to us. The reward model is initialized from the pretrained LIV [9] ResNet-50 checkpoint, with the full visual backbone finetuned during reward learning. We use RGB observations from the base-view camera.

The training objective combines the VIP temporal loss with simple goal contrastive learning: it increases similarity between successful goals from different demonstrations and decreases similarity between successful goals and online failure states. Successful goals and negative examples are sampled from the terminal segments of offline successful demonstrations and online failed episodes, respectively. The model is first finetuned on offline demonstrations and then updated online as the policy collects new experience.

For each episode, we sample a successful goal frame g and keep it fixed. Let $c_\theta(s, g)$ denote cosine similarity in the learned representation space, and define $\Phi_{\text{GCR}}(s; g) = (c_\theta(s, g) + 1)/2$. The online reward follows the difference form and reward scaling described in the paper:

$$R_{\text{GCR}}(s, a, s') = 10(R_{\text{sparse}}(s, a, s') + \Phi_{\text{GCR}}(s'; g) - \Phi_{\text{GCR}}(s; g)). \quad (15)$$

We score the current and next observations with the same predictor snapshot. Replayed expert demonstrations retain only the sparse environment reward, scaled by the same factor of 10.

B Additional Mathematical Interpretations

Mathematically, the precise landscape topography across these situations is regulated by the concentration parameter κ . The parameter κ functions exactly as the inverse temperature in the standard LogSumExp operator [43], which controls the smoothness of the maximum operator over the support sets. Under the hard-assignment limit where the representation distributions become highly concentrated ($\kappa \rightarrow \infty$), the smooth expectation is dominated entirely by the single nearest neighbor in each buffer, simplifying HiRE into a clear geometric subtraction: $\lim_{\kappa \rightarrow \infty} \Phi_{\text{HiRE}}(s) = \max_{g^+ \in \mathcal{B}^+} \text{sim}_\phi(s, g^+) - \lambda \max_{g^- \in \mathcal{B}^-} \text{sim}_\phi(s, g^-)$. While this hard limit provides an intuitive visualization of the underlying boundary subtraction, our final implementation utilizes the smooth version ($\kappa < \infty$). This choice delivers denser, continuous gradients and leverages the full empirical data distribution within the interaction history to stabilize policy finetuning.

C Additional Results

C.1 Ablation Results

We organize the ablations into four groups: (i) **foundation representation**, swapping the encoder to evaluate HiRE with different foundation representations; (ii) **hyperparameter sensitivity**, varying the contrastive weight λ and negative-buffer capacity to examine HiRE’s insensitivity to changes in these settings; (iii) **reward composition**, comparing full HiRE with direct addition of the edited potential (without PBRS) and direct addition of raw goal similarity (without hindsight editing or PBRS); (iv) **buffer design**, disabling the negative buffer to measure how much the failure data contributes to reward correction on top of the success data.

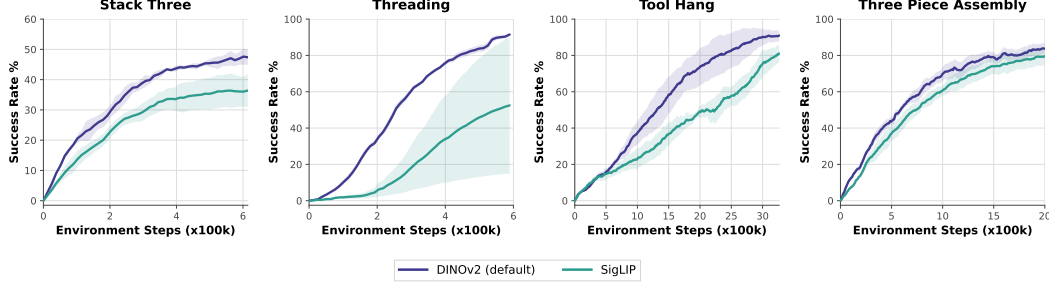


Figure 8: **Foundation representation ablations.** We compare HiRE with frozen DINOv2 and SigLIP encoders. Curves show mean success rates, with shaded regions denoting variability across available random seeds.

Foundation Representation Model. HiRE only requires a frozen encoder ϕ that maps visual observations to patch-level features, so it can accommodate different foundation representations. We replace the default DINOv2 encoder with the SigLIP visual encoder [11] (ViT-B/16), retaining patch-averaged cosine similarity and hindsight contrastive editing. As shown in Fig. 8, **both encoders provide useful dense guidance across all four tasks**, while DINOv2 consistently achieves higher success rates. The difference is task-dependent: SigLIP approaches DINOv2 on Tool_Hang and Three_Piece_Assembly, but exhibits slower improvement on Stack_Three and particularly Threading, where its variability across seeds is also larger. These results suggest that HiRE is versatile across foundation representations, while the choice of encoder still affects sample efficiency and the reliability of the resulting reward landscape.

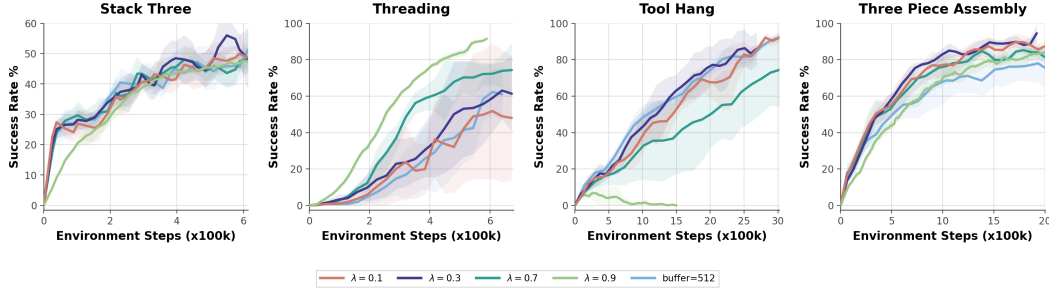


Figure 9: **Hyperparameter sensitivity across four simulation tasks.** We compare contrastive weights $\lambda \in \{0.1, 0.3, 0.7, 0.9\}$ and an additional configuration with negative-buffer capacity 512 (buffer=512). Curves are averaged over three random seeds, with shaded regions denoting variability across seeds.

Hyperparameter Sensitivity. We evaluate sensitivity by varying the contrastive weight $\lambda \in \{0.1, 0.3, 0.7, 0.9\}$ and testing a negative-buffer capacity of 512. As shown in Fig. 9, **HiRE retains effective learning across a broad range of settings** on Stack_Three and Three_Piece_Assembly. The variants perform similarly on Stack_Three, while all configurations achieve substantial performance improvement on Three_Piece_Assembly, only with some differences in learning speed and asymptotic performance. The high-precision Threading task shows greater sensitivity: reducing λ or using the larger buffer slows improvement relative to the default setting. A larger buffer can retain outdated failures and weaken the focus on current traps, while a smaller λ may provide insufficient repulsion when the failure distribution is broad. The Tool_Hang sweep reveals a complementary regime: $\lambda = 0.1$ and 0.3 provide stronger guidance, whereas $\lambda = 0.9$ fails to sustain improvement. Here, it echoes what we reveal in Fig. 5: the positive distribution remains highly concentrated, whereas the negative one is much wider and scattered, probably because the failure cases vary, but the successful paradigm is very unique. This empirical finding confirms assigning an even larger concentration factor κ^+ than to κ^- , which we discuss in Sec. 4.1. Overall, the results support broad parameter insensitivity on the multi-stage tasks, while highlighting the need for appropriate failure-based correction on precision-critical tasks.

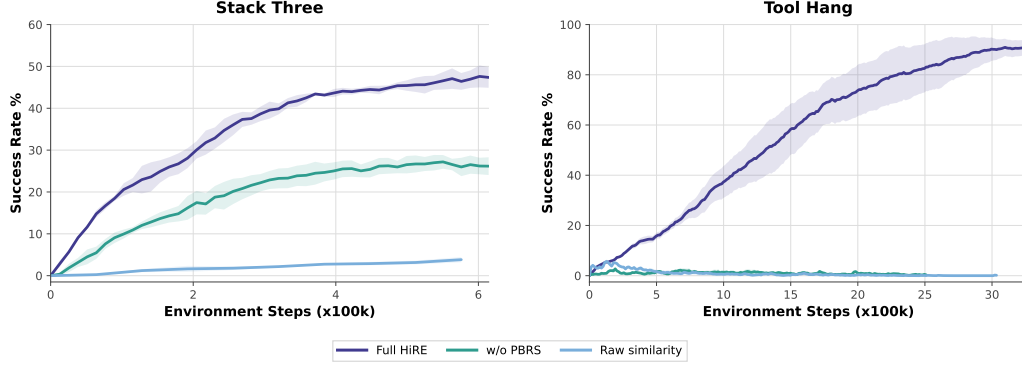


Figure 10: **Reward-composition comparisons on Stack_Three and Tool_Hang.** We compare full HiRE, direct addition of the HiRE potential (w/o PBRS), and direct addition of raw goal similarity (Raw similarity). Curves are averaged over three random seeds, with shaded regions denoting variability across seeds.

Reward Composition. The full HiRE reward combines hindsight contrastive editing (Eq. 5) with potential-based reward shaping (Eq. 6). Fig. 10 compares this composition with two alternative reward forms:

- **HiRE without PBRS.** We add the edited potential directly to the sparse success reward:

$$R(s, a, s') = R_{\text{sparse}}(s, a, s') + w \cdot \Phi_{\text{HiRE}}(s'). \quad (16)$$

This retains hindsight contrastive editing but removes the temporal-difference structure of PBRS.

- **Raw similarity.** We replace the edited potential with the raw goal-similarity potential $\Phi_{\text{raw}}(s) = \text{sim}_{\phi}(s, g_{\star}^+)$, where $g_{\star}^+$ is the final successful frame of an expert demonstration, and add it directly to the sparse reward:

$$R(s, a, s') = R_{\text{sparse}}(s, a, s') + w \cdot \Phi_{\text{raw}}(s'). \quad (17)$$

This uses a fixed visual goal without hindsight correction from successful and failed rollouts or the temporal-difference structure of PBRS.

Here w denotes the dense-reward coefficient; the equations summarize the reward forms. The compared runs also differ in reward scaling and replay settings, so these results compare the resulting reward configurations rather than isolating a single implementation change.

On `Stack_Three`, **full HiRE achieves both faster improvement and higher success rates** than the two alternatives. Removing PBRS leads to slower improvement and an earlier plateau, while raw similarity provides little learning progress. The advantage of the variant without PBRS over raw similarity suggests that hindsight editing provides useful guidance even with direct reward addition. The further gain from full HiRE is consistent with the benefit of integrating the edited potential through temporal differences that cancels intermediate terms during intrinsic reward accumulation.

The distinction is sharper on `Tool_Hang`, where both alternative configurations remain near zero while full HiRE continues to improve. These results suggest that **combining hindsight editing with PBRS is especially valuable for contact-rich, high-precision manipulation**: direct addition of either the edited potential or raw goal similarity does not reproduce the learning progress achieved by full HiRE.

Buffer Design. Section 4.1 interprets HiRE as a contrast between two empirical manifolds: $\mathcal{B}^+$ provides success-dominated attraction that rescues false negatives, whereas $\mathcal{B}^-$ provides failure-dominated repulsion against false-positive trap states. To isolate the role of the latter, we evaluate a **positive-buffer-only** variant that removes the negative buffer and keeps only attraction toward the success manifold:

$$\Phi(s) = \mathbb{L}_{\mathcal{B}^+}^{\kappa^+}(\text{sim}_{\phi}(s, g^+)). \quad (18)$$

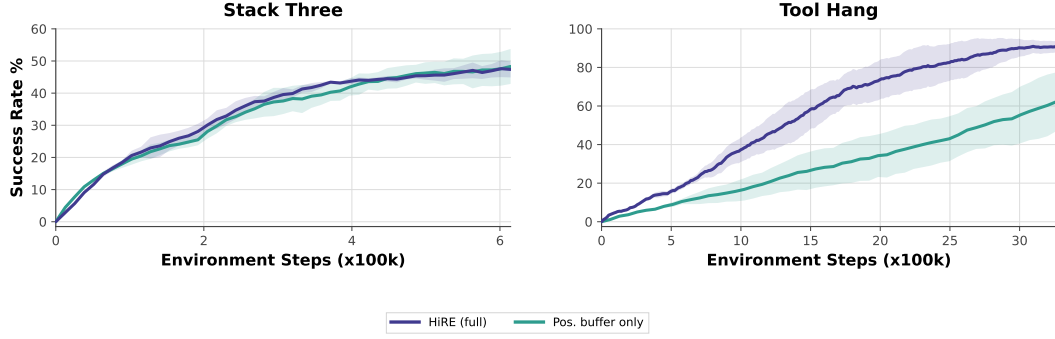


Figure 11: **Buffer-design ablations on `Stack_Three` and `Tool_Hang`.** We compare full HiRE with the positive-buffer-only variant, which removes the negative buffer and keeps only success-manifold attraction. Curves are averaged over three random seeds, with shaded regions denoting variability across seeds.

This ablation tests whether the success manifold alone is sufficient, and how much additional benefit comes from explicit failure awareness. As shown in Fig. 11, the positive-buffer-only variant closely tracks full HiRE on `Stack_Three`, and finishes slightly higher with overlapping variability bands. This suggests that success-manifold attraction supplies much of the dense guidance on this task. The benefit of the full contrastive reward is clearer on `Tool_Hang`. This task is contact-rich, high-precision, and multi-stage, so it produces many near-goal failures that appear visually successful. Such states are exactly the false-positive traps that attraction to $\mathcal{B}^+$ cannot distinguish, but that repulsion from $\mathcal{B}^-$ can suppress. Full HiRE improves faster and reaches above 90%, while the positive-only variant improves more slowly and achieves a lower final success rate. This difference across the two tasks suggests that the value of the negative buffer grows with task difficulty and the prevalence of visually deceptive failures. Overall, the ablation is consistent with the interpretation in Section 4.1: the positive buffer supplies attraction toward successful states, while the negative buffer provides the complementary correction needed to suppress visually deceptive failures.